

To achieve a fully functional, enterprise-grade architecture using **Spring Cloud OpenFeign** for cross-service communication, we must explicitly map out the internal endpoints and client interfaces.

By separating files into independent service boundaries, we eliminate dependency cycles. When Service A needs data from Service B, it calls a `@FeignClient` mapping rather than querying Service B's database directly.

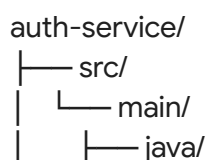
Here is the complete, file-by-file directory architecture for all services, including OpenFeign clients and standard application.properties configurations.

1. api-gateway

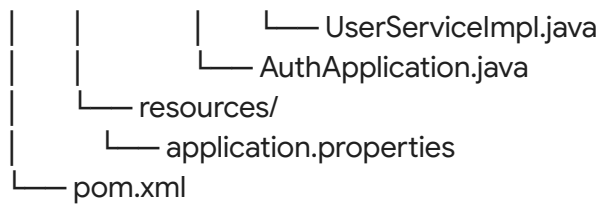
```
api-gateway/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── microlend/
│   │   │   │   │   ├── gateway/
│   │   │   │   │   │   ├── config/
│   │   │   │   │   │   │   ├── GatewayConfig.java
│   │   │   │   │   │   │   └── ApiGatewayApplication.java
│   │   │   │   │   │   └──
│   │   │   │   │   └──
│   │   │   │   └──
│   │   │   └──
│   │   └── resources/
│   │       ├── application.properties
│   └── pom.xml
```

2. config-server

```
config-server/
├── src/
│   ├── main/
│   │   ├── java/
```

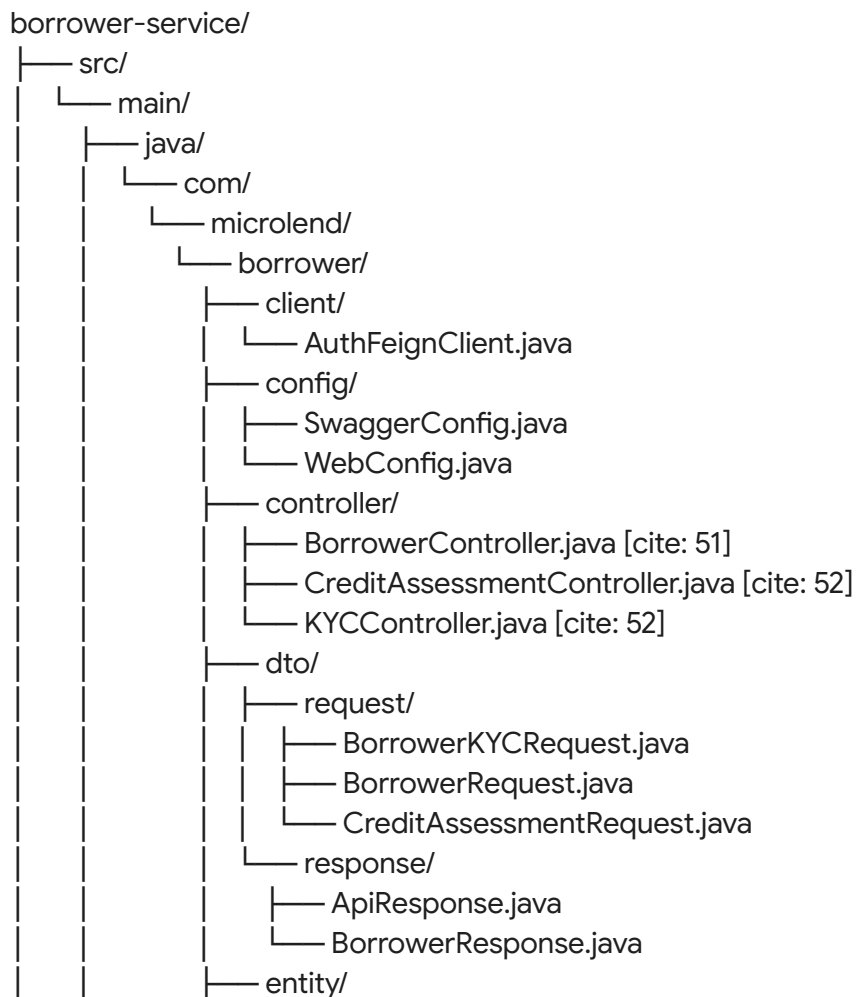


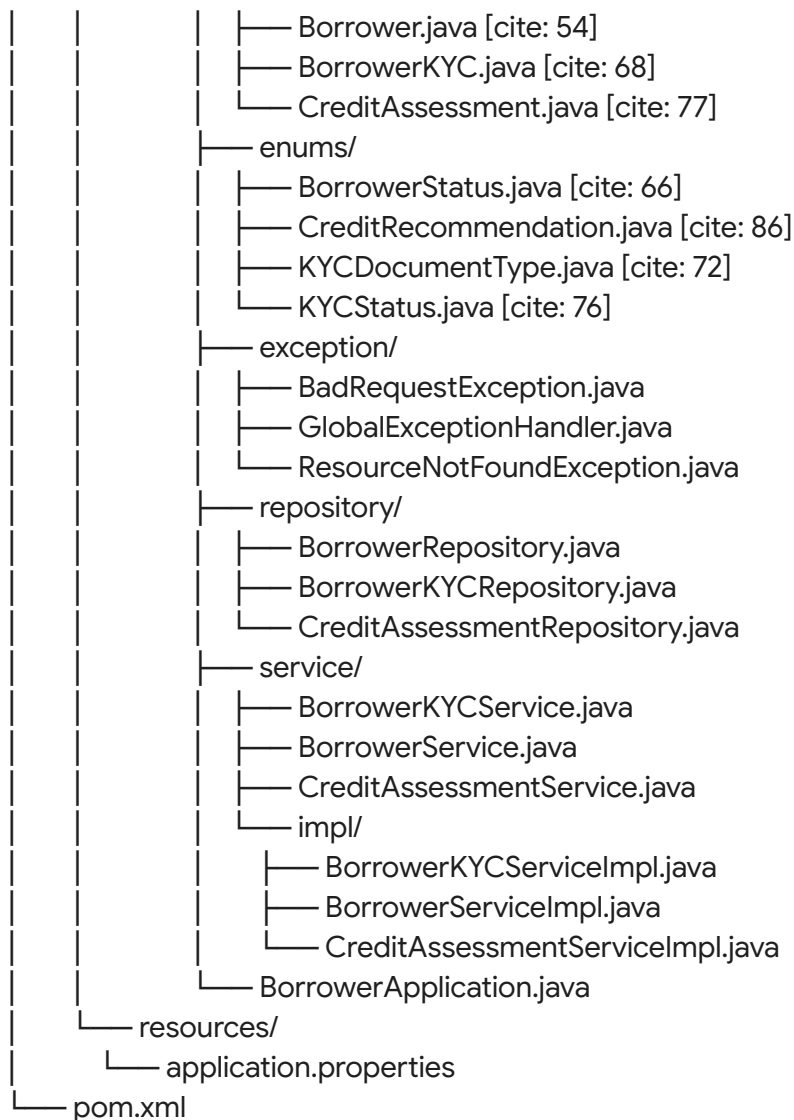
```
└─ com/
  └─ microlend/
    └─ auth/
      └─ config/
        ├── DataSeeder.java
        ├── SecurityConfig.java
        └─ SwaggerConfig.java
      └─ controller/
        ├── AuditLogController.java [cite: 32]
        ├── AuthController.java [cite: 31]
        └─ UserController.java [cite: 31]
      └─ dto/
        └─ request/
          ├── LoginRequest.java
          └─ RegisterUserRequest.java
        └─ response/
          ├── ApiResponse.java
          ├── AuthResponse.java
          └─ UserResponse.java
      └─ entity/
        ├── AuditLog.java [cite: 43]
        └─ User.java [cite: 34]
      └─ enums/
        ├── UserRole.java [cite: 38]
        └─ UserStatus.java [cite: 42]
      └─ exception/
        ├── BadRequestException.java
        ├── GlobalExceptionHandler.java
        └─ ResourceNotFoundException.java
      └─ repository/
        ├── AuditLogRepository.java
        └─ UserRepository.java
      └─ security/
        ├── CustomUserDetailsService.java
        ├── JwtAuthFilter.java
        └─ JwtUtil.java
      └─ service/
        ├── AuditLogService.java
        ├── AuthService.java
        ├── UserService.java
        └─ impl/
          ├── AuditLogServiceImpl.java
          └─ AuthServiceImpl.java
```



5. borrower-service

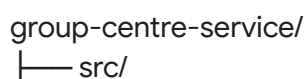
Manages core credit profiling and KYC. It contains an OpenFeign client to securely communicate with the auth-service to cross-verify the registration of system operators (VerifiedByID).

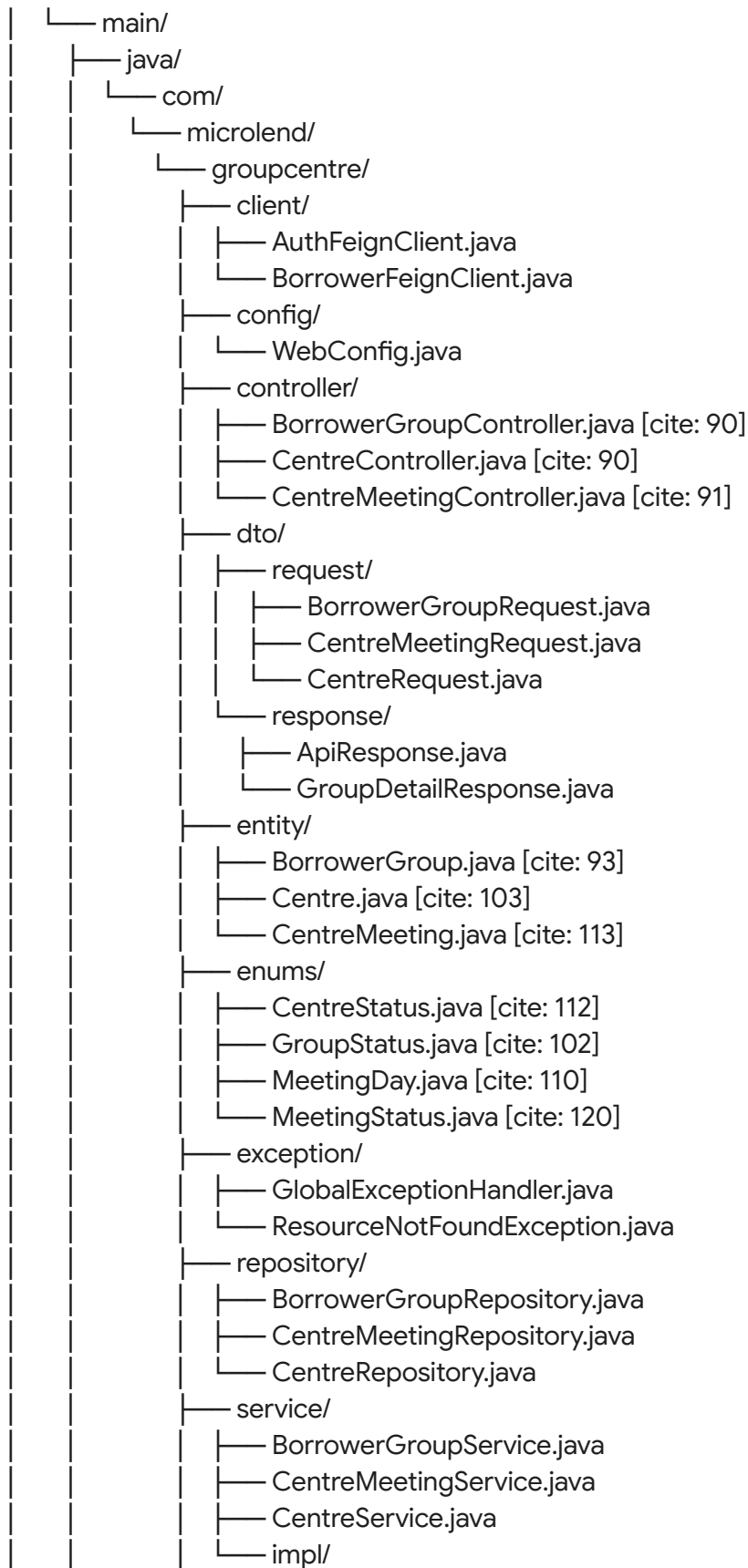


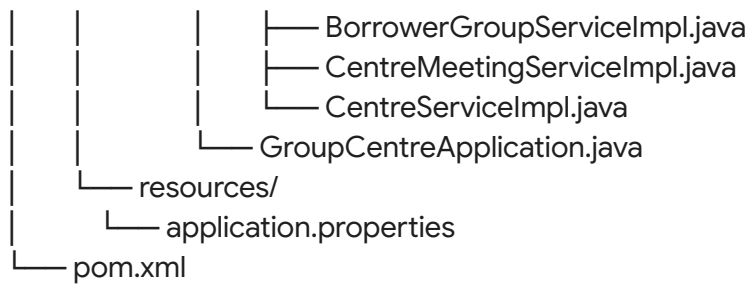


6. group-centre-service

Coordinates joint-liability configurations. It relies on OpenFeign clients to safely query borrower-service data (verifying individual borrower statuses before adding them to a cluster group) and auth-service configurations (confirming that an assigned FieldOfficerID exists).

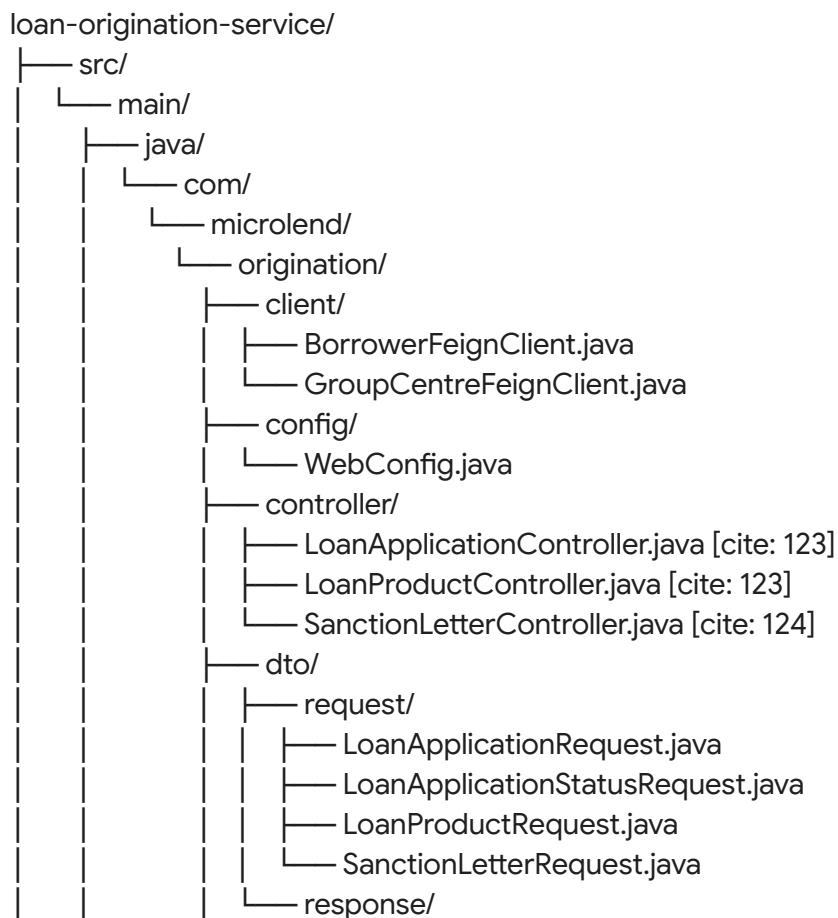


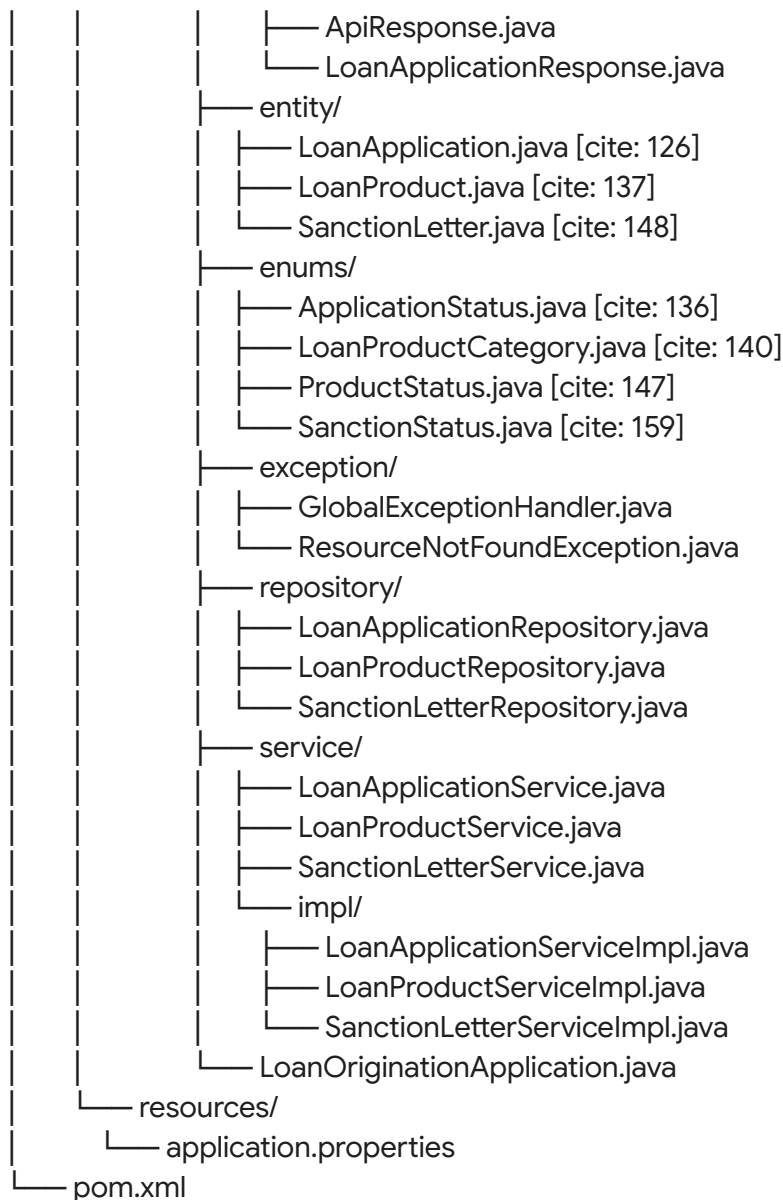




7. loan-origination-service

Determines initial applicant underwriting conditions. It integrates Feign clients to confirm borrower internal ratings via borrower-service and check structural group balances via group-centre-service before issuing a sanction document.





8. loan-account-service

Maintains live portfolio balances and amortization rules. It monitors incoming underwriting workflows via loan-origination-service using Feign loops to convert approved applications into active loan accounts.

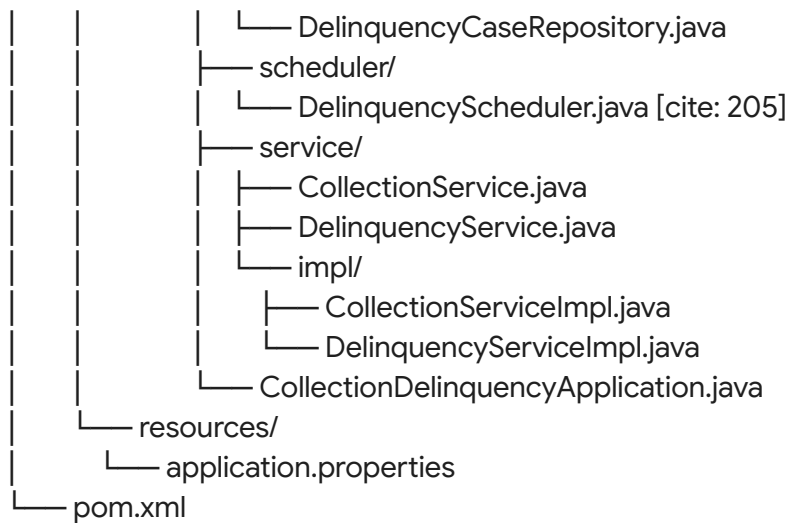
```
loan-account-service/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── microlend/
│   │   │   │   │   ├── account/
│   │   │   │   │   │   ├── client/
│   │   │   │   │   │   │   ├── OriginationFeignClient.java
│   │   │   │   │   │   ├── config/
│   │   │   │   │   │   │   ├── WebConfig.java
│   │   │   │   │   │   ├── controller/
│   │   │   │   │   │   │   ├── LoanAccountController.java [cite: 161]
│   │   │   │   │   │   ├── dto/
│   │   │   │   │   │   │   ├── request/
│   │   │   │   │   │   │   │   ├── DisbursementRequest.java
│   │   │   │   │   │   │   ├── response/
│   │   │   │   │   │   │   │   ├── ApiResponse.java
│   │   │   │   │   │   │   │   ├── AccountScheduleResponse.java
│   │   │   │   │   │   ├── entity/
│   │   │   │   │   │   │   ├── LoanAccount.java [cite: 164]
│   │   │   │   │   │   │   ├── RepaymentSchedule.java [cite: 176]
│   │   │   │   │   │   ├── enums/
│   │   │   │   │   │   │   ├── InstallmentStatus.java [cite: 186]
│   │   │   │   │   │   │   ├── InterestType.java [cite: 145]
│   │   │   │   │   │   │   ├── LoanAccountStatus.java [cite: 175]
│   │   │   │   │   │   ├── exception/
│   │   │   │   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   │   │   │   ├── ResourceNotFoundException.java
│   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   │   ├── LoanAccountRepository.java
│   │   │   │   │   │   │   ├── RepaymentScheduleRepository.java
│   │   │   │   │   │   ├── service/
│   │   │   │   │   │   │   ├── LoanDisbursementService.java
│   │   │   │   │   │   │   ├── impl/
│   │   │   │   │   │   │   │   ├── LoanDisbursementServiceImpl.java
│   │   │   │   │   │   └── LoanAccountApplication.java
│   │   │   └── resources/
│   │   │       ├── application.properties
│   │   └── pom.xml
```

9. collection-delinquency-service

Records transaction histories and runs background delinquency monitoring. It updates payment details directly via loan-account-service using OpenFeign.

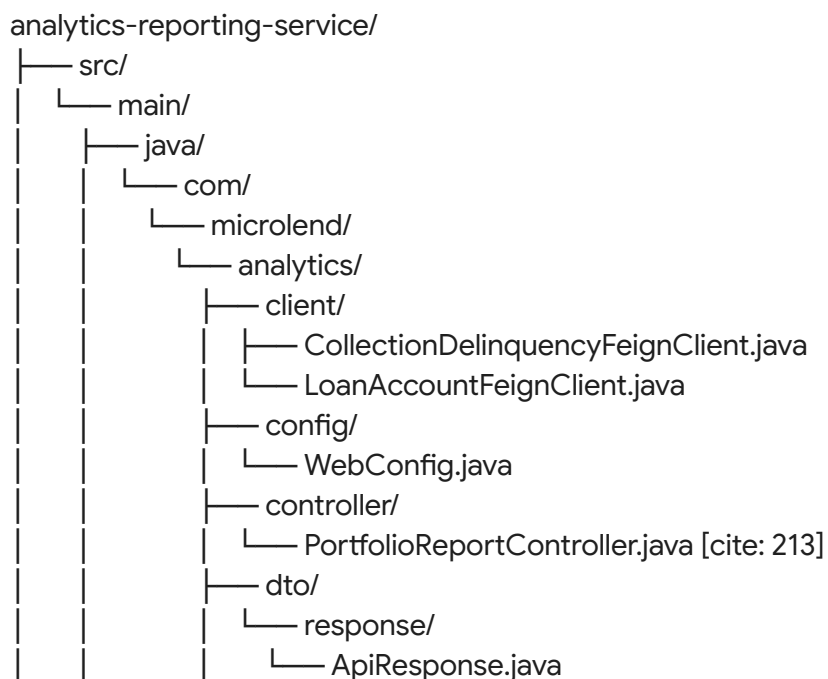
collection-delinquency-service/

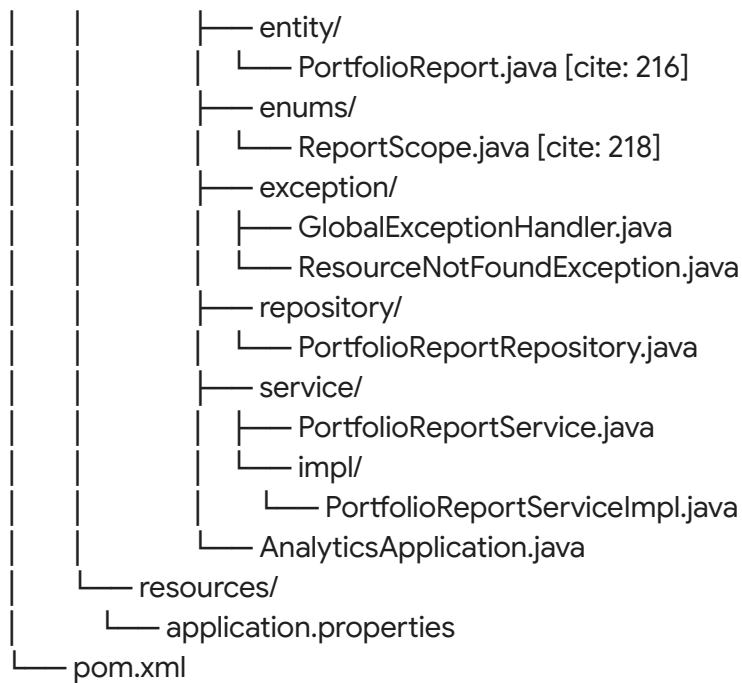
```
|— src/
|   |— main/
|   |   |— java/
|   |   |   |— com/
|   |   |   |   |— microlend/
|   |   |   |   |   |— collection/
|   |   |   |   |   |   |— client/
|   |   |   |   |   |   |   |— LoanAccountFeignClient.java
|   |   |   |   |   |   |— config/
|   |   |   |   |   |   |   |— WebConfig.java
|   |   |   |   |   |   |— controller/
|   |   |   |   |   |   |   |— CollectionController.java [cite: 189]
|   |   |   |   |   |   |   |— DelinquencyController.java [cite: 190]
|   |   |   |   |   |   |— dto/
|   |   |   |   |   |   |   |— request/
|   |   |   |   |   |   |   |   |— CollectionRequest.java
|   |   |   |   |   |   |   |   |— DelinquencyCaseRequest.java
|   |   |   |   |   |   |   |— response/
|   |   |   |   |   |   |   |   |— ApiResponse.java
|   |   |   |   |   |   |— entity/
|   |   |   |   |   |   |   |— CollectionRecord.java [cite: 192]
|   |   |   |   |   |   |   |— DelinquencyCase.java [cite: 201]
|   |   |   |   |   |   |— enums/
|   |   |   |   |   |   |   |— CollectionMode.java [cite: 199]
|   |   |   |   |   |   |   |— CollectionStatus.java [cite: 200]
|   |   |   |   |   |   |   |— DelinquencyAction.java [cite: 209]
|   |   |   |   |   |   |   |— DelinquencyStatus.java [cite: 210]
|   |   |   |   |   |   |   |— PARBucket.java [cite: 206]
|   |   |   |   |   |   |— exception/
|   |   |   |   |   |   |   |— GlobalExceptionHandler.java
|   |   |   |   |   |   |   |— ResourceNotFoundException.java
|   |   |   |   |   |   |— repository/
|   |   |   |   |   |   |   |— CollectionRecordRepository.java
```



10. analytics-reporting-service

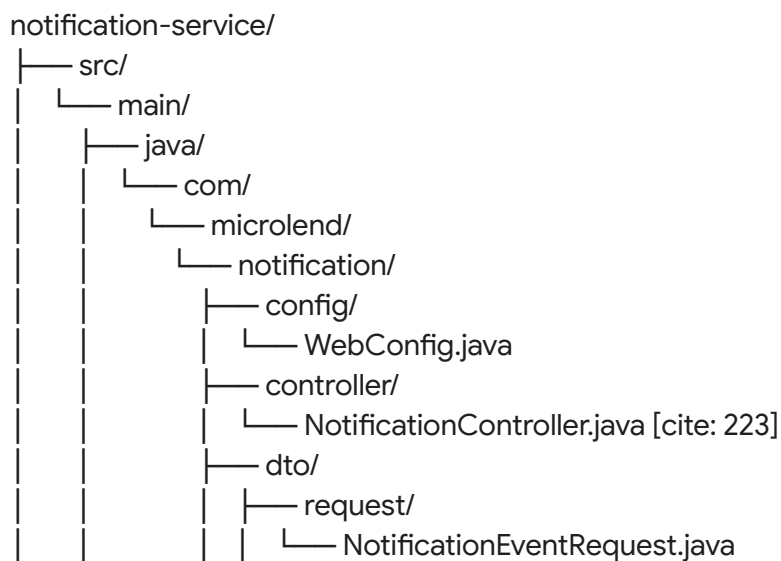
Consolidates platform wide metrics. It leverages OpenFeign clients to dynamically gather details from loan-account-service and collection-delinquency-service without storing duplicate transactional records.

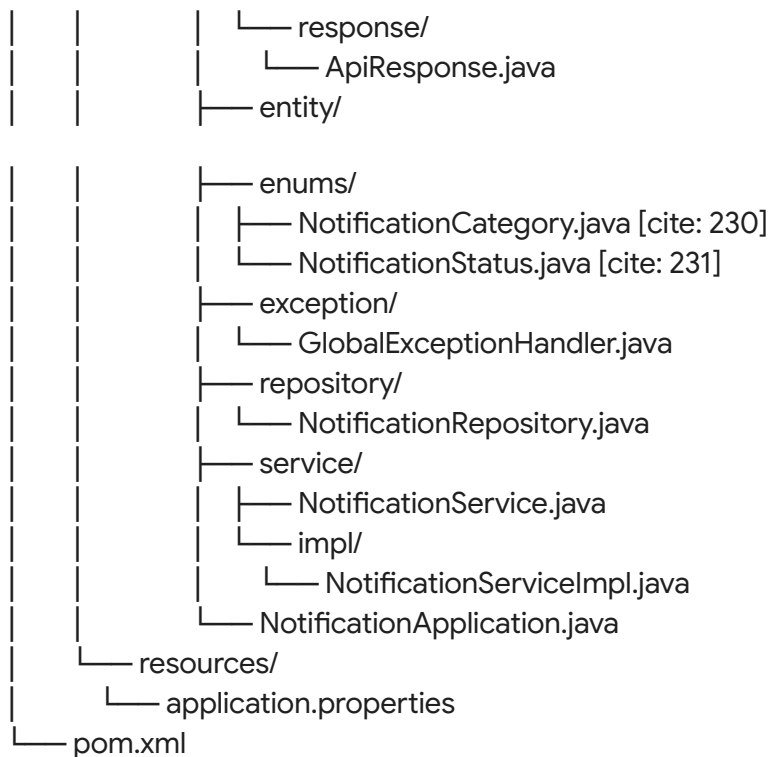




11. notification-service

An isolated delivery utility. Other components issue non blocking POST requests via OpenFeign to trigger notification event entries.





Example Implementations: OpenFeign in Action

To ensure you can implement this immediately, here is exactly how your Feign Client interfaces and properties look:

Example Feign Client Interface (borrower-service)

```
package com.microlend.borrower.client;

import com.microlend.borrower.dto.response.UserResponse;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "auth-service", path = "/api/v1/users")
public interface AuthFeignClient {
```

```
@GetMapping("/{id}")
    getUserById @PathVariable("id")
    ;
}
```

Example application.properties configuration (borrower-service)

```
# Microservice Identifiers
spring.application.name=borrower-service
server.port=8082

# Eureka Client Registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# Spring Cloud Config Hook
spring.config.import=optional:configserver:http://localhost:8888

# Database Configs
spring.datasource.url=jdbc:postgresql://localhost:5432/microlend_borrower
spring.datasource.username=postgres
spring.datasource.password=securepassword
spring.jpa.hibernate.ddl-auto=update

# OpenFeign & Resilience configurations
feign.client.config.default.connect-timeout=5000
feign.client.config.default.read-timeout=5000
```